

Test 1 ADP470S 9 April 2026

Question 1 (20)

- 1.1 Trace the execution of the Selection Sort algorithm on the following array: [34, 12, 5, 8, 20]. Show the state of the array after each successful swap and indicate which elements are considered "sorted" at each step. [6]
- 1.2 The Quick Sort algorithm relies on a partitioning step. Using the last element as the pivot, demonstrate the partitioning process for the array: [15, 3, 9, 21, 12, 18, 10]. Show the movements of the pointers (typically i and j) and the final position of the pivot after the partition is complete. [8]
- 1.3 Compare Bubble Sort and Insertion Sort. While both have a worst-case time complexity of $O(n^2)$, explain why Insertion Sort is generally more efficient for "nearly sorted" datasets. [6]

Question 2 (20)

- 2.1 Explain the recursive definition of the Factorial function ($n!$). Trace the execution for $n=4$, explicitly showing each recursive call and how the values are returned up the call chain to produce the final result. [7]
- 2.2 The Fibonacci sequence is defined as $F(n)=F(n-1)+F(n-2)$. Draw the recursion tree for the calculation of $F(4)$. Based on this tree, explain why the naive recursive approach is considered inefficient in terms of the number of function calls. Hint: show the Big-O expression for time complexity. [7]
- 2.3 Provide a pseudocode or programming language implementation for an iterative version of the Fibonacci sequence. Briefly explain how this approach avoids the overhead associated with the recursive version. [6]

Question 3 (20)

- 3.1 Analyze the Quick Sort algorithm in terms of space complexity. Although it is an in-place sort, it requires additional memory for the recursion stack. Explain why its average-case space complexity is $O(\log n)$ and identify the scenario that leads to its worst-case space complexity of $O(n)$. [8]
- 3.2 Create a table comparing the Time Complexity (Best, Average, and Worst cases) for the following four sorting algorithms: [12]
Bubble Sort
Insertion Sort
Selection Sort
Quick Sort

Question 4 (20)

- 4.1 Explain the Big-O notation and explain its importance. Why do we focus on the "worst-case" scenario rather than the "best-case" when analyzing the performance or scalability of an algorithm? [6]
- 4.2 Compare the Space Complexity of a recursive Factorial algorithm with an iterative Factorial algorithm. Explain why one typically requires $O(n)$ space while the other requires $O(1)$ space. [8]
- 4.3 Describe the relationship between recursion and the call stack. What happens to the call stack if a recursive function fails to reach its base case, and what is the technical term for this error? [6]